DM层搭建

1. 销售主题宽表

销售主题宽表，基于日统计值，统计出年、月、周的数据。

指标和dws一致。

1.1 建表

```
--  建库
create database IF NOT EXISTS yp_dm;


DROP TABLE IF EXISTS yp_dm.dm_sale;
CREATE TABLE yp_dm.dm_sale(
   time_type string COMMENT '统计时间维度：year、month、week、date',
   year_code string COMMENT '年code',
   year_month string COMMENT '年月',
   month_code string COMMENT '月份编码',
   day_month_num string COMMENT '一月第几天',
   dim_date_id string COMMENT '日期',
   year_week_name_cn string COMMENT '年中第几周',

   group_type string COMMENT '分组类型：store,trade_area,city,brand,min_class,mid_class,max_class,
all',
   province_id string COMMENT '省份id',
   province_name string COMMENT '省份名称',
   city_id string COMMENT '城市id',
   city_name string COMMENT '城市name',
   trade_area_id string COMMENT '商圈id',
   trade_area_name string COMMENT '商圈名称',
   store_id string COMMENT '店铺的id',
   store_name string COMMENT '店铺名称',
   brand_id string COMMENT '品牌id',
   brand_name string COMMENT '品牌名称',
   max_class_id string COMMENT '商品大类id',
   max_class_name string COMMENT '大类名称',
   mid_class_id string COMMENT '中类id',
```

```
  mid_class_name string COMMENT '中类名称',
  min_class_id string COMMENT '小类id',
  min_class_name string COMMENT '小类名称',
  --  =======统计=======
  --   销售收入
  sale_amt DECIMAL(38,4) COMMENT '销售收入',
  --   平台收入
  plat_amt DECIMAL(38,4) COMMENT '平台收入',
  -- 配送成交额
  deliver_sale_amt DECIMAL(38,2) COMMENT '配送成交额',
  -- 小程序成交额
  mini_app_sale_amt DECIMAL(38,2) COMMENT '小程序成交额',
  -- 安卓APP成交额
  android_sale_amt DECIMAL(38,2) COMMENT '安卓APP成交额',
  --  苹果APP成交额
  ios_sale_amt DECIMAL(38,2) COMMENT '苹果APP成交额',
  -- PC商城成交额
  pcweb_sale_amt DECIMAL(38,2) COMMENT 'PC商城成交额',
  -- 成交单量
  order_cnt BIGINT COMMENT '成交单量',
  -- 参评单量
  eva_order_cnt BIGINT COMMENT '参评单量comment=>cmt',
  -- 差评单量
  bad_eva_order_cnt BIGINT COMMENT '差评单量negtive-comment=>ncmt',
  -- 配送成交单量
  deliver_order_cnt BIGINT COMMENT '配送单量',
  -- 退款单量
  refund_order_cnt BIGINT COMMENT '退款单量',
  -- 小程序成交单量
  miniapp_order_cnt BIGINT COMMENT '小程序成交单量',
  -- 安卓APP订单量
  android_order_cnt BIGINT COMMENT '安卓APP订单量',
  -- 苹果APP订单量
  ios_order_cnt BIGINT COMMENT '苹果APP订单量',
  -- PC商城成交单量
  pcweb_order_cnt BIGINT COMMENT 'PC商城成交单量'
)
COMMENT '销售主题宽表'
-- 统计日期,不能用来分组统计
PARTITIONED BY(date_time STRING)
ROW format delimited fields terminated BY '\t'
stored AS orc tblproperties ('orc.compress' = 'SNAPPY');
```

1.2 导入数据

1.2.1 grouping_id

```
grouping(dc.store_id, dc.trade_area_id, dc.city_id, dc.brand_id, dc.min_class_id,
dc.mid_class_id, dc.max_class_id) grouping_id
```

二进制转换逻辑

store：0001111->15

trade_area：1001111->79

city：1101111->111

brand：1110111->119

min_class：1111000->120

mid_class：1111100->124

max_class：1111110->126

all：1111111->127

1.2.2 实现

```
insert into yp_dm.dm_sale
-- 获取日期数据（周、月的环比/同比日期）
with dt1 as (
  select
   dim_date_id, date_code
    ,date_id_mom -- 与本月环比的上月日期
    ,date_id_mym -- 与本月同比的上年日期
    ,year_code
    ,month_code
    ,year_month    --年月
    ,day_month_num --几号
    ,week_day_code --周几
    ,year_week_name_cn  --年周
from yp_dwd.dim_date
),
groupby as (
    select
    -- 统计日期
       '2021-08-31' as date_time,
    -- 时间维度      year、month、date
```

```
      case when grouping(dt1.year_code, dt1.month_code, dt1.day_month_num, dt1.dim_date_id) =
0
        then 'date'
         when grouping(dt1.year_code, dt1.year_week_name_cn) = 0
        then 'week'
        when grouping(dt1.year_code, dt1.month_code, dt1.year_month) = 0
        then 'month'
        when grouping(dt1.year_code) = 0
        then 'year'
      end
      as time_type,
      dt1.year_code,
      dt1.year_month,
      dt1.month_code,
      dt1.day_month_num, --几号
      dt1.dim_date_id,
       dt1.year_week_name_cn,  --第几周
    -- 产品维度类型: store, trade_area, city, brand, min_class, mid_class, max_class, all
      CASE WHEN grouping(dc.store_id)=0
        THEN 'store'
          WHEN grouping(dc.trade_area_id)=0
          THEN 'trade_area'
          WHEN grouping(dc.city_id)=0
          THEN 'city'
          WHEN grouping(dc.brand_id)=0
          THEN 'brand'
          WHEN grouping(dc.min_class_id)=0
          THEN 'min_class'
          WHEN grouping(dc.mid_class_id)=0
          THEN 'mid_class'
          WHEN grouping(dc.max_class_id)=0
          THEN 'max_class'
          ELSE 'all'
      END
      as group_type_new,
      grouping(dc.store_id, dc.trade_area_id, dc.city_id, dc.brand_id, dc.min_class_id,
dc.mid_class_id, dc.max_class_id) grouping_id,
      group_type group_type_old,

dc.province_id,dc.province_name,dc.city_id,dc.city_name,dc.trade_area_id,dc.trade_area_name,d
c.store_id,dc.store_name,dc.brand_id,dc.brand_name,dc.max_class_id,dc.max_class_name,dc.mid_c
lass_id,dc.mid_class_name,dc.min_class_id,dc.min_class_name,
    -- 统计值
```

```
       sum(dc.sale_amt) as sale_amt,
       sum(dc.plat_amt) as plat_amt,
       sum(dc.deliver_sale_amt) as deliver_sale_amt,
       sum(dc.mini_app_sale_amt) as mini_app_sale_amt,
       sum(dc.android_sale_amt) as android_sale_amt,
       sum(dc.ios_sale_amt) as ios_sale_amt,
       sum(dc.pcweb_sale_amt) as pcweb_sale_amt,

       sum(dc.order_cnt) as order_cnt,
       sum(dc.eva_order_cnt) as eva_order_cnt,
       sum(dc.bad_eva_order_cnt) as bad_eva_order_cnt,
       sum(dc.deliver_order_cnt) as deliver_order_cnt,
       sum(dc.refund_order_cnt) as refund_order_cnt,
       sum(dc.miniapp_order_cnt) as miniapp_order_cnt,
       sum(dc.android_order_cnt) as android_order_cnt,
       sum(dc.ios_order_cnt) as ios_order_cnt,
       sum(dc.pcweb_order_cnt) as pcweb_order_cnt
    from yp_dws.dws_sale_daycount dc
       left join dt1 on dc.dt = dt1.date_code
    --WHERE dc.dt >= '2021-08-31'
    group by
    grouping sets (
    -- 年，注意养成加小括号的习惯
       (dt1.year_code, group_type),
       (dt1.year_code, city_id, city_name, province_id, province_name, group_type),
       (dt1.year_code, city_id, city_name, province_id, province_name, trade_area_id,
trade_area_name, group_type),
       (dt1.year_code, city_id, city_name, province_id, province_name, trade_area_id,
trade_area_name, store_id, store_name, group_type),
       (dt1.year_code, brand_id, brand_name, group_type),
       (dt1.year_code, max_class_id, max_class_name, group_type),
       (dt1.year_code, max_class_id, max_class_name,mid_class_id, mid_class_name, group_type),
       (dt1.year_code, max_class_id, max_class_name,mid_class_id, mid_class_name,min_class_id,
min_class_name, group_type),
    -- 月
       (dt1.year_code, dt1.month_code, dt1.year_month, group_type),
       (dt1.year_code, dt1.month_code, dt1.year_month, city_id, city_name, province_id,
province_name, group_type),
       (dt1.year_code, dt1.month_code, dt1.year_month, city_id, city_name, province_id,
province_name, trade_area_id, trade_area_name, group_type),
       (dt1.year_code, dt1.month_code, dt1.year_month, city_id, city_name, province_id,
province_name, trade_area_id, trade_area_name, store_id, store_name, group_type),
       (dt1.year_code, dt1.month_code, dt1.year_month, brand_id, brand_name, group_type),
```

```
        (dt1.year_code, dt1.month_code, dt1.year_month, max_class_id, max_class_name,
group_type),
        (dt1.year_code, dt1.month_code, dt1.year_month, max_class_id,
max_class_name,mid_class_id, mid_class_name, group_type),
        (dt1.year_code, dt1.month_code, dt1.year_month, max_class_id,
max_class_name,mid_class_id, mid_class_name,min_class_id, min_class_name, group_type),
    -- 日
        (dt1.year_code, dt1.month_code, dt1.day_month_num, dt1.dim_date_id, group_type),
        (dt1.year_code, dt1.month_code, dt1.day_month_num, dt1.dim_date_id, city_id, city_name,
province_id, province_name, group_type),
        (dt1.year_code, dt1.month_code, dt1.day_month_num, dt1.dim_date_id, city_id, city_name,
province_id, province_name, trade_area_id, trade_area_name, group_type),
        (dt1.year_code, dt1.month_code, dt1.day_month_num, dt1.dim_date_id, city_id, city_name,
province_id, province_name, trade_area_id, trade_area_name, store_id, store_name, group_type),
        (dt1.year_code, dt1.month_code, dt1.day_month_num, dt1.dim_date_id, brand_id, brand_name,
group_type),
        (dt1.year_code, dt1.month_code, dt1.day_month_num, dt1.dim_date_id, max_class_id,
max_class_name, group_type),
        (dt1.year_code, dt1.month_code, dt1.day_month_num, dt1.dim_date_id, max_class_id,
max_class_name,mid_class_id, mid_class_name, group_type),
        (dt1.year_code, dt1.month_code, dt1.day_month_num, dt1.dim_date_id, max_class_id,
max_class_name,mid_class_id, mid_class_name,min_class_id, min_class_name, group_type),
    -- 周
        (dt1.year_code, dt1.year_week_name_cn, group_type),
        (dt1.year_code, dt1.year_week_name_cn, city_id, city_name, province_id, province_name,
group_type),
        (dt1.year_code, dt1.year_week_name_cn, city_id, city_name, province_id, province_name,
trade_area_id, trade_area_name, group_type),
        (dt1.year_code, dt1.year_week_name_cn, city_id, city_name, province_id, province_name,
trade_area_id, trade_area_name, store_id, store_name, group_type),
        (dt1.year_code, dt1.year_week_name_cn, brand_id, brand_name, group_type),
        (dt1.year_code, dt1.year_week_name_cn, max_class_id, max_class_name, group_type),
        (dt1.year_code, dt1.year_week_name_cn, max_class_id, max_class_name,mid_class_id,
mid_class_name, group_type),
        (dt1.year_code, dt1.year_week_name_cn, max_class_id, max_class_name,mid_class_id,
mid_class_name,min_class_id, min_class_name, group_type)
    )
    -- order by time_type desc
)
select
    -- 时间维度      year、month、date
    time_type,
    year_code,
    year_month,
    month_code,
    day_month_num, --几号
```

```
       dim_date_id,
       year_week_name_cn,  --第几周
    -- 产品维度类型: store, trade_area, city, brand, min_class, mid_class, max_class, all
       group_type_new as group_type,
--       业务属性维度

province_id,province_name,city_id,city_name,trade_area_id,trade_area_name,store_id,store_name,
brand_id,brand_name,max_class_id,max_class_name,mid_class_id,mid_class_name,min_class_id,min_
class_name,
--       订单金额
       sale_amt, plat_amt, deliver_sale_amt, mini_app_sale_amt, android_sale_amt, ios_sale_amt,
pcweb_sale_amt,
--       订单量
       order_cnt, eva_order_cnt, bad_eva_order_cnt, deliver_order_cnt, refund_order_cnt,
miniapp_order_cnt, android_order_cnt, ios_order_cnt, pcweb_order_cnt,
    -- 统计日期
       date_time
from groupby
where group_type_new=group_type_old
-- order by time_type, year_code, year_month, dim_date_id, year_week_name_cn, sale_amt desc,
order_cnt desc
;
```

1.3 循环导入

虽然销售DWS只循环计算昨天的数据，但销售DM要重新计算整年的数据，因此需要将昨天所在整年的数据都包含进来重新计算。

```
delete from yp_dws.dws_sale_daycount where date_time='${TD_DATE}';


select ...,
   '${TD_DATE}' as date_time
from yp_dws.dws_sale_daycount dc
   left join dt1 on dc.dt = dt1.date_code
-- 重新计算整年的数据
WHERE dc.dt >= '${TD_DATE_YEAR}-01-01'
group by
grouping sets ...
```

2. 商品主题宽表

商品SKU主题宽表，需求指标和dws层一致，但不是每日统计数据，而是总累计值 和 月度累计值，可以基于DWS日统计数据计算。

2.1 建表

```
drop table if exists yp_dm.dm_sku;
create table yp_dm.dm_sku
(
   time_type string COMMENT '统计时间维度：all、month',
   year_code string COMMENT '年code',
   year_month string COMMENT '年月',
   sku_id string comment 'sku_id',
   sku_name string comment '商品名称',
   order_count bigint comment '被下单次数',
   order_num bigint comment '被下单件数',
   order_amount decimal(38,2) comment '被下单金额',
   payment_count  bigint  comment '被支付次数',
   payment_num bigint comment '被支付件数',
   payment_amount  decimal(38,2) comment '被支付金额',
   refund_count bigint comment '退款次数',
   refund_num bigint comment '退款件数',
   refund_amount decimal(38,2) comment '退款金额',
   cart_count bigint comment '被加入购物车次数',
   cart_num bigint comment '被加入购物车件数',
   favor_count bigint comment '被收藏次数',
   evaluation_good_count bigint comment '好评数',
   evaluation_mid_count bigint comment '中评数',
   evaluation_bad_count bigint comment '差评数'
)
COMMENT '商品主题宽表'
-- 统计日期,不能用来分组统计
PARTITIONED BY(date_time STRING)
ROW format delimited fields terminated BY '\t'
stored AS orc tblproperties ('orc.compress' = 'SNAPPY');
```

2.2 首次导入

当主题需求计算历史累积值时，首次全量SQL需要计算所有的历史数据，后续增量SQL只需要累加新数据即可。

首次执行需要从dws日统计表求出总累计值。

```
insert into yp_dm.dm_sku
with all_count as (
    select
        'all' time_type,
        null year_code,
        null year_month,
        sku_id, max(sku_name),
        sum(coalesce(order_count,0)) as order_count,
        sum(coalesce(order_num,0)) as order_num,
        sum(coalesce(order_amount,0)) as order_amount,
        sum(coalesce(payment_count,0)) payment_count,
        sum(coalesce(payment_num,0)) payment_num,
        sum(coalesce(payment_amount,0)) payment_amount,
        sum(coalesce(refund_count,0)) refund_count,
        sum(coalesce(refund_num,0)) refund_num,
        sum(coalesce(refund_amount,0)) refund_amount,
        sum(coalesce(cart_count,0)) cart_count,
        sum(coalesce(cart_num,0)) cart_num,
        sum(coalesce(favor_count,0)) favor_count,
        sum(coalesce(evaluation_good_count,0))   evaluation_good_count,
        sum(coalesce(evaluation_mid_count,0))    evaluation_mid_count,
        sum(coalesce(evaluation_bad_count,0))    evaluation_bad_count
    from yp_dws.dws_sku_daycount
--     where order_count > 0
    group by sku_id
),
month as (
    select
        'month' time_type,
        substring(dt, 1, 4) year_code,
        substring(dt, 1, 7) year_month,
        sku_id,  max(sku_name),
        sum(coalesce(order_count,0)) as order_count,
```

```
        sum(coalesce(order_num,0)) as order_num,
        sum(coalesce(order_amount,0)) as order_amount,
        sum(coalesce(payment_count,0)) payment_count,
        sum(coalesce(payment_num,0)) payment_num,
        sum(coalesce(payment_amount,0)) payment_amount,
        sum(coalesce(refund_count,0)) refund_count,
        sum(coalesce(refund_num,0)) refund_num,
        sum(coalesce(refund_amount,0)) refund_amount,
        sum(coalesce(cart_count,0)) cart_count,
        sum(coalesce(cart_num,0)) cart_num,
        sum(coalesce(favor_count,0)) favor_count,
        sum(coalesce(evaluation_good_count,0))   evaluation_good_count,
        sum(coalesce(evaluation_mid_count,0))    evaluation_mid_count,
        sum(coalesce(evaluation_bad_count,0))    evaluation_bad_count
    from yp_dws.dws_sku_daycount
    group by sku_id, substring(dt, 1, 7), substring(dt, 1, 4)
)
select ac.*, '2021-08-31' date_time
from all_count ac
union all
select m.*, '2021-08-31' date_time
from month m;
```

2.3 循环导入

后续执行不需要计算所有历史数据，直接在之前的累计值基础上累加新数据即可。

累加值=旧的累积值+昨天新数据；

月份数据直接分组聚合即可。

2.3.1 建立临时表(使用hive执行)

```
drop table if exists yp_dm.dm_sku_tmp;
create table yp_dm.dm_sku_tmp
(
    time_type string COMMENT '统计时间维度：all、month',
    year_code string COMMENT '年code',
    year_month string COMMENT '年月',
    sku_id string comment 'sku_id',
```

```
    sku_name string comment '商品名称',
    order_count bigint comment '被下单次数',
    order_num bigint comment '被下单件数',
    order_amount decimal(38,2) comment '被下单金额',
    payment_count   bigint  comment '被支付次数',
    payment_num bigint comment '被支付件数',
    payment_amount  decimal(38,2) comment '被支付金额',
    refund_count bigint comment '退款次数',
    refund_num bigint comment '退款件数',
    refund_amount decimal(38,2) comment '退款金额',
    cart_count bigint comment '被加入购物车次数',
    cart_num bigint comment '被加入购物车件数',
    favor_count bigint comment '被收藏次数',
    evaluation_good_count bigint comment '好评数',
    evaluation_mid_count bigint comment '中评数',
    evaluation_bad_count bigint comment '差评数'
)
COMMENT '商品主题宽表'
-- 统计日期,不能用来分组统计
PARTITIONED BY(date_time STRING)
ROW format delimited fields terminated BY '\t'
stored AS orc tblproperties ('orc.compress' = 'SNAPPY');
```

2.3.2 合并新旧数据

后续执行不需要计算所有历史数据，直接在之前的累计值基础上累加新数据即可。

累加值=旧的累积值+昨天新数据；

月份数据直接分组聚合即可。

```
--2、合并新旧数据
-- 增量
insert into yp_dm.dm_sku_tmp
--     dm旧数据
with max_time as (
    select max(date_time) dt from yp_dm.dm_sku
),
old as (
```

```
    select
        *
    from yp_dm.dm_sku, max_time
--  读取最后一次计算的总累计值
    where date_time=dt and time_type='all'
),
--  昨天 的dws新数据
last_day as (
    select
        sku_id,
        max(sku_name) sku_name,
        sum(coalesce(order_count,0)) as order_count,
        sum(coalesce(order_num,0)) as order_num,
        sum(coalesce(order_amount,0)) as order_amount,
        sum(coalesce(payment_count,0)) payment_count,
        sum(coalesce(payment_num,0)) payment_num,
        sum(coalesce(payment_amount,0)) payment_amount,
        sum(coalesce(refund_count,0)) refund_count,
        sum(coalesce(refund_num,0)) refund_num,
        sum(coalesce(refund_amount,0)) refund_amount,
        sum(coalesce(cart_count,0)) cart_count,
        sum(coalesce(cart_num,0)) cart_num,
        sum(coalesce(favor_count,0)) favor_count,
        sum(coalesce(evaluation_good_count,0))   evaluation_good_count,
        sum(coalesce(evaluation_mid_count,0))    evaluation_mid_count,
        sum(coalesce(evaluation_bad_count,0))    evaluation_bad_count
    from yp_dws.dws_sku_daycount
--   昨天的数据
    where dt='${TD_DATE}'
    group by sku_id
),
-- 总累积数值=旧数据+昨日新数据
all as (
    select
        'all' time_type,
        null year_code,
        null year_month,
        coalesce(last_day.sku_id,old.sku_id) sku_id,
        coalesce(last_day.sku_name,old.sku_name) sku_name,
    --        订单 累积历史数据
        coalesce(old.order_count,0) + coalesce(last_day.order_count,0) order_count,
```

```
        coalesce(old.order_num,0) + coalesce(last_day.order_num,0) order_num,
        coalesce(old.order_amount,0) + coalesce(last_day.order_amount,0) order_amount,
    --      支付单 累积历史数据
        coalesce(old.payment_count,0) + coalesce(last_day.payment_count,0) payment_count,
        coalesce(old.payment_num,0) + coalesce(last_day.payment_count,0) payment_num,
        coalesce(old.payment_amount,0) + coalesce(last_day.payment_count,0) payment_amount,
    --      退款单 累积历史数据
        coalesce(old.refund_count,0) + coalesce(last_day.refund_count,0) refund_count,
        coalesce(old.refund_num,0) + coalesce(last_day.refund_num,0) refund_num,
        coalesce(old.refund_amount,0) + coalesce(last_day.refund_amount,0) refund_amount,
    --      购物车 累积历史数据
        coalesce(old.cart_count,0) + coalesce(last_day.cart_count,0) cart_count,
        coalesce(old.cart_num,0) + coalesce(last_day.cart_num,0) cart_num,
    --      收藏 累积历史数据
        coalesce(old.favor_count,0) + coalesce(last_day.favor_count,0) favor_count,
    --      评论 累积历史数据
        coalesce(old.evaluation_good_count,0) + coalesce(last_day.evaluation_good_count,0)
evaluation_good_count,
        coalesce(old.evaluation_mid_count,0) + coalesce(last_day.evaluation_mid_count,0)
evaluation_mid_count,
        coalesce(old.evaluation_bad_count,0) + coalesce(last_day.evaluation_bad_count,0)
evaluation_bad_count
    from old
    full join last_day on old.sku_id=last_day.sku_id
),
-- 月统计值
last_month as (
    select
        'month' time_type,
        substring(dt, 1, 4) year_code,
        substring(dt, 1, 7) year_month,
        sku_id, max(sku_name),
        sum(coalesce(order_count,0)) as order_count,
        sum(coalesce(order_num,0)) as order_num,
        sum(coalesce(order_amount,0)) as order_amount,
        sum(coalesce(payment_count,0)) payment_count,
        sum(coalesce(payment_num,0)) payment_num,
        sum(coalesce(payment_amount,0)) payment_amount,
        sum(coalesce(refund_count,0)) refund_count,
        sum(coalesce(refund_num,0)) refund_num,
        sum(coalesce(refund_amount,0)) refund_amount,
        sum(coalesce(cart_count,0)) cart_count,
```

```
        sum(coalesce(cart_num,0)) cart_num,
        sum(coalesce(favor_count,0)) favor_count,
        sum(coalesce(evaluation_good_count,0))   evaluation_good_count,
        sum(coalesce(evaluation_mid_count,0))    evaluation_mid_count,
        sum(coalesce(evaluation_bad_count,0))    evaluation_bad_count
    from yp_dws.dws_sku_daycount
--   上个月的数据
    where dt>=concat(substring('${TD_DATE}', 1, 7), '-01')
    group by sku_id, substring(dt, 1, 7), substring(dt, 1, 4)
)
-- 合并总累积和月度数据
select *, '${TD_DATE}' date_time from all
union all
select *, '${TD_DATE}' date_time from last_month;
```

2.3.3 临时表覆盖宽表

```
delete from yp_dm.dm_sku where date_time='${TD_DATE}';
insert into yp_dm.dm_sku
select * from yp_dm.dm_sku_tmp;
```

3. 用户主题宽表

用户主题宽表，需求指标和dws层一致，但不是每日统计数据，而是总累计值 和 月份累计值，可以基于DWS日统计数据计算。

3.1 建表

```
drop table if exists yp_dm.dm_user;
create table yp_dm.dm_user
(
    time_type string COMMENT '统计时间维度：all、month',
    year_code string COMMENT '年code',
    year_month string COMMENT '年月',
    user_id string  comment '用户id',
    -- 登录
    login_date_first string  comment '首次登录日期',
```

```
    login_date_last string  comment '末次登录日期',
    login_count bigint comment '登录天数',
    -- 购物车
    cart_date_first string comment '首次加入购物车日期',
    cart_date_last string comment '末次加入购物车日期',
    cart_count bigint comment '加入购物车次数',
    cart_amount decimal(38,2) comment '加入购物车金额',
    -- 订单
    order_date_first string  comment '首次下单日期',
    order_date_last string  comment '末次下单日期',
    order_count bigint comment '下单次数',
    order_amount decimal(38,2) comment '下单金额',
    -- 支付
    payment_date_first string  comment '首次支付日期',
    payment_date_last string  comment '末次支付日期',
    payment_count bigint comment '支付次数',
    payment_amount decimal(38,2) comment '支付金额'
)
COMMENT '用户主题宽表'
-- 统计日期,不能用来分组统计
PARTITIONED BY(date_time STRING)
ROW format delimited fields terminated BY '\t'
stored AS orc tblproperties ('orc.compress' = 'SNAPPY');
```

3.2 首次导入

```
-- 全量
insert into yp_dm.dm_user
-- 登录次数、首末日期
with login_count as (
    select
        min(dt) as login_date_first,
        max (dt) as login_date_last,
        sum(if(login_count>0,1,0)) as login_count,
       user_id
    from yp_dws.dws_user_daycount
    where login_count > 0
    group by user_id
```

```
),
-- 购物车次数、金额、首末日期
cart_count as (
    select
        min(dt) as cart_date_first,
        max(dt) as cart_date_last,
        sum(coalesce(cart_count, 0)) as cart_count,
        sum(coalesce(cart_amount, 0)) as cart_amount,
        user_id
    from yp_dws.dws_user_daycount
    where cart_count > 0
    group by user_id
),
-- 订单量、订单额、首末日期
order_count as (
    select
        min(dt) as order_date_first,
        max(dt) as order_date_last,
        sum(coalesce(order_count, 0)) as order_count,
        sum(coalesce(order_amount, 0)) as order_amount,
        user_id
    from yp_dws.dws_user_daycount
    where order_count > 0
    group by user_id
),
-- 支付单量、支付金额、首末日期
payment_count as (
    select
        min(dt) as payment_date_first,
        max(dt) as payment_date_last,
        sum(coalesce(payment_count, 0)) as payment_count,
        sum(coalesce(payment_amount, 0)) as payment_amount,
        user_id
    from yp_dws.dws_user_daycount
    where payment_count > 0
    group by user_id
),
fulljoin as (
    select
        'all' time_type,
        null year_code,
```

```
      null year_month,
      '2021-08-31' date_time,
      coalesce(l.user_id, o.user_id, p.user_id, cc.user_id) user_id,
   --   登录
      l.login_date_first,
      l.login_date_last,
      l.login_count,
   --   购物车
      cc.cart_date_first,
      cc.cart_date_last,
      cc.cart_count,
      cc.cart_amount,
   --   订单
      o.order_date_first,
      o.order_date_last,
      o.order_count,
      o.order_amount,
   --   支付
      p.payment_date_first,
      p.payment_date_last,
      p.payment_count,
      p.payment_amount
   from login_count l
   full join order_count o on l.user_id=o.user_id
   full join payment_count p on o.user_id=p.user_id
   full join cart_count cc on p.user_id=cc.user_id
),
all as (
select
      'all' time_type,
      null year_code,
      null year_month,
      user_id,
   --   登录
      min(login_date_first) login_date_first,
      max(login_date_last) login_date_last,
      sum(coalesce(login_count, 0)) login_count,
   --   购物车
      min(cart_date_first) cart_date_first,
      max(cart_date_last) cart_date_last,
      sum(coalesce(cart_count, 0)) cart_count,
```

```
        sum(coalesce(cart_amount, 0)) cart_amount,
    --    订单
        min(order_date_first) order_date_first,
        max(order_date_last) order_date_last,
        sum(coalesce(order_count, 0)) order_count,
        sum(coalesce(order_amount, 0)) order_amount,
    --    支付
        min(payment_date_first) payment_date_first,
        max(payment_date_last) payment_date_last,
        sum(coalesce(payment_count, 0)) payment_count,
        sum(coalesce(payment_amount, 0)) payment_amount
    from fulljoin
    group by user_id
),
-- select * from all order by user_id, login_count desc, cart_amount desc, order_count desc,
payment_count desc;
-- 月份数据统计
month as (
    select
        'month' time_type,
        substring(dt, 1, 4) year_code,
        substring(dt, 1, 7) year_month,
        user_id,
        --    登录
        min(if(login_count>0, dt, null)) login_date_first,
        max(if(login_count>0, dt, null)) login_date_last,
        sum(if(login_count>0,1,0)) login_count,
        --    购物车
        min(if(cart_count>0, dt, null)) cart_date_first,
        max(if(cart_count>0, dt, null)) cart_date_last,
        sum(if(login_count>0, coalesce(cart_count,0), 0)) cart_count,
        sum(if(login_count>0, coalesce(cart_amount,0), 0)) cart_amount,
        --    订单
        min(if(order_count>0, dt, null)) order_date_first,
        max(if(order_count>0, dt, null)) order_date_last,
        sum(if(order_count>0, coalesce(order_count,0), 0)) order_count,
        sum(if(order_count>0, coalesce(order_amount,0), 0)) order_amount,
        --    支付
        min(if(payment_count>0, dt, null)) payment_date_first,
        max(if(payment_count>0, dt, null)) payment_date_last,
        sum(if(payment_count>0, coalesce(payment_count,0), 0)) payment_count,
```

```
        sum(if(payment_count>0, coalesce(payment_amount,0), 0)) payment_amount
    from yp_dws.dws_user_daycount
    group by user_id, substring(dt, 1, 7), substring(dt, 1, 4)
)
select *, '2021-08-31' date_time from all
union all
select *, '2021-08-31' date_time from month
;
```

3.3 循环导入

后续执行不需要计算所有历史数据，直接在之前的累计值基础上累加新数据即可。

累加值=旧的累积值+昨天的数据；月份数据直接分组聚合即可。

3.3.1 建立临时表（hive执行）

```
drop table if exists yp_dm.dm_user_tmp;
create table yp_dm.dm_user_tmp
(
    time_type string COMMENT '统计时间维度：all、month',
    year_code string COMMENT '年code',
    year_month string COMMENT '年月',
    user_id string  comment '用户id',
    -- 登录
    login_date_first string  comment '首次登录日期',
    login_date_last string  comment '末次登录日期',
    login_count bigint comment '登录天数',
    -- 购物车
    cart_date_first string comment '首次加入购物车日期',
    cart_date_last string comment '末次加入购物车日期',
    cart_count bigint comment '加入购物车次数',
    cart_amount decimal(38,2) comment '加入购物车金额',
    -- 订单
    order_date_first string  comment '首次下单日期',
    order_date_last string  comment '末次下单日期',
```

```
    order_count bigint comment '下单次数',
    order_amount decimal(38,2) comment '下单金额',
    -- 支付
    payment_date_first string  comment '首次支付日期',
    payment_date_last string  comment '末次支付日期',
    payment_count bigint comment '支付次数',
    payment_amount decimal(38,2) comment '支付金额'
)
COMMENT '用户主题宽表'
-- 统计日期,不能用来分组统计
PARTITIONED BY(date_time STRING)
ROW format delimited fields terminated BY '\t'
stored AS orc tblproperties ('orc.compress' = 'SNAPPY');
```

3.3.2 合并新旧数据

后续执行不需要计算所有历史数据，直接在之前的累计值基础上累加新数据即可。

累加值=旧的累积值+昨天的数据；月份数据直接分组聚合即可。

```
-- 增量
insert into yp_dm.dm_user_tmp
with max_time as(
    select max(date_time) dt from yp_dm.dm_user
),
old as (
    select * from yp_dm.dm_user, max_time
--  读取最后一次计算的总累计值
    where date_time=dt and time_type='all'
),
last_day as (
    select
        user_id,
--      登录次数
        login_count,
--      收藏
        store_collect_count,
        goods_collect_count,
--      购物车
        cart_count,
```

```
      cart_amount,
--    订单
      order_count,
      order_amount,
--    支付
      payment_count,
      payment_amount
   from yp_dws.dws_user_daycount
--  昨天新数据
   where dt='${TD_DATE}'
),
all as (
--  累加
   select
      'all' time_type,
      null year_code,
      null year_month,
      coalesce(last_day.user_id,old.user_id) user_id,
   --    登录
      if(old.login_date_first is null and
last_day.login_count>0,'${TD_DATE}',old.login_date_first) login_date_first,
      if(last_day.login_count>0,'${TD_DATE}',old.login_date_last) login_date_last,
      coalesce(old.login_count,0)+if(last_day.login_count>0,1,0) login_count,
   --    购物车
      if(old.cart_date_first is null and last_day.cart_count>0,'${TD_DATE}',old.cart_date_first)
cart_date_first,
      if(last_day.cart_count>0,'${TD_DATE}',old.cart_date_last) cart_date_last,
      coalesce(old.cart_count,0)+coalesce(last_day.cart_count,0) cart_count,
      coalesce(old.cart_amount,0)+coalesce(last_day.cart_amount,0) cart_amount,
   --    订单
      if(old.order_date_first is null and
last_day.order_count>0,'${TD_DATE}',old.order_date_first) order_date_first,
      if(last_day.order_count>0,'${TD_DATE}',old.order_date_last) order_date_last,
      coalesce(old.order_count,0)+coalesce(last_day.order_count,0) order_count,
      coalesce(old.order_amount,0)+coalesce(last_day.order_amount,0) order_amount,
   --    支付
      if(old.payment_date_first is null and
last_day.payment_count>0,'${TD_DATE}',old.payment_date_first) payment_date_first,
      if(last_day.payment_count>0,'${TD_DATE}',old.payment_date_last) payment_date_last,
      coalesce(old.payment_count,0)+coalesce(last_day.payment_count,0) payment_count,
      coalesce(old.payment_amount,0)+coalesce(last_day.payment_amount,0) payment_amount
   from old
```

```
    full join last_day on old.user_id=last_day.user_id
),
last_month as (
    select
        'month' time_type,
        substring(dt, 1, 4) year_code,
        substring(dt, 1, 7) year_month,
        user_id,
        --    登录
        min(if(login_count>0, dt, null)) login_date_first,
        max(if(login_count>0, dt, null)) login_date_last,
        sum(if(login_count>0,1,0)) login_count,
        --    购物车
        min(if(cart_count>0, dt, null)) cart_date_first,
        max(if(cart_count>0, dt, null)) cart_date_last,
        sum(if(login_count>0,coalesce(cart_count, 0),0)) cart_count,
        sum(if(login_count>0,coalesce(cart_amount, 0),0)) cart_amount,
        --    订单
        min(if(order_count>0, dt, null)) order_date_first,
        max(if(order_count>0, dt, null)) order_date_last,
        sum(if(order_count>0,coalesce(order_count, 0),0)) order_count,
        sum(if(order_count>0,coalesce(order_amount, 0),0)) order_amount,
        --    支付
        min(if(payment_count>0, dt, null)) payment_date_first,
        max(if(payment_count>0, dt, null)) payment_date_last,
        sum(if(payment_count>0,coalesce(payment_count, 0),0)) payment_count,
        sum(if(payment_count>0,coalesce(payment_amount, 0),0)) payment_amount
    from yp_dws.dws_user_daycount
--   上个月数据
    where dt>=concat(substring('${TD_DATE}', 1, 7), '-01')
    group by user_id, substring(dt, 1, 7), substring(dt, 1, 4)
)
select *, '${TD_DATE}' date_time from all
union all
select *, '${TD_DATE}' date_time from last_month;
```

3.3.3 临时表覆盖宽表

```
delete from yp_dm.dm_user where date_time='${TD_DATE}';
insert into yp_dm.dm_user
```

```
select * from yp_dm.dm_user_tmp;
```